

# Code Study Guide For The Relational Experiments

---

The most important files are:

```
generate_minimal_symmetry_dataset.py
generate_nontransitive_symmetry_dataset.py
generate_minimal_antisymmetry_dataset.py
generate_sparse_supervised_by_dataset.py
run_minimal_symmetry_tfm_experiment.py
run_kge_relation_baselines.py
```

The most important result files are:

```
results_symmetry_tfms/metrics.csv
results_symmetry_kge/metrics.csv
results_non_transitive_symmetry_kge/metrics.csv
results_antisymmetry_tfms/metrics.csv
results_antisymmetry_kge/metrics.csv
results_sparse_supervised_by_tfms/metrics.csv
results_sparse_kge/metrics.csv
```

## 1. The Shared Data Contract

All experiments ultimately produce a model-facing CSV with this schema:

```
entity_1,entity_2,label,split
```

This is the only table consumed by the TFM runner and the KGE runner.

The code rule is:

```
generators create labels and splits
runners consume labels and splits
```

Therefore, when studying a result, always inspect two code paths:

```
generator code path:
  how were labels created?
  how were train/validation/test rows assigned?

runner code path:
  how were features encoded?
```

```
how was the model fit?  
how were probabilities thresholded?  
how were metrics computed?
```

## 2. TFM Runner Code Path

File:

```
run_minimal_symmetry_tfm_experiment.py
```

Despite the filename, this runner is used for symmetry, total-order antisymmetry, and sparse `supervised_by`, because all datasets share the same four-column schema.

### 2.1 Data Loading

Function:

```
load_dataset_rows()  
line: run_minimal_symmetry_tfm_experiment.py:145
```

This is where the CSV enters the experiment.

The function:

- opens `--data-path`;
- reads rows with `csv.DictReader`;
- requires `entity_1`, `entity_2`, `label`, `split`;
- rejects obvious leakage columns;
- checks labels are binary;
- checks train and test are non-empty;
- checks train and test contain both labels.

Why this matters:

```
If this function passes, the model-facing CSV has the minimum valid  
structure for the experiment.
```

What it does not check:

```
It does not prove that the scientific split is strong.  
It only proves the table is structurally valid.
```

### 2.2 Feature Construction

Function:

```
make_feature_frame()  
line: run_minimal_symmetry_tfm_experiment.py:183
```

This function creates the model input features.

It keeps only:

```
entity_1  
entity_2
```

It does not include:

```
label  
split  
hidden_group_id  
hidden_seniority_rank  
role  
student_entity  
supervisor_entity
```

The categorical dtype uses a shared entity vocabulary:

```
entity_categories = all entities appearing in entity_1 or entity_2
```

Code reference:

```
run_minimal_symmetry_tfm_experiment.py:500
```

Meaning:

```
the same entity receives the same categorical code in train, validation,  
and test
```

This is appropriate for these same-entity-universe experiments.

It would need reconsideration for a new-entity generalization protocol.

## 2.3 TabPFN Encoding

Function:

```
encode_for_tabpfn()  
line: run_minimal_symmetry_tfm_experiment.py:193
```

TabPFN receives two integer-coded categorical columns:

```
column 0 = code(entity_1)  
column 1 = code(entity_2)
```

The model is told these columns are categorical:

```
categorical_features_indices=[0, 1]
```

Code reference:

```
run_minimal_symmetry_tfm_experiment.py:209
```

This matters because entity IDs are arbitrary symbols. They should not be interpreted as numeric magnitudes.

## 2.4 TabICL Encoding

Function:

```
build_tabicl_classifier()  
line: run_minimal_symmetry_tfm_experiment.py:219
```

TabICL receives the pandas categorical feature frame directly.

The fit path is shared with TabPFN inside:

```
fit_predict_one_model()  
line: run_minimal_symmetry_tfm_experiment.py:414
```

## 2.5 TFM Fit And Prediction

Function:

```
fit_predict_one_model()  
line: run_minimal_symmetry_tfm_experiment.py:414
```

This is the core TFM computation function.

Important lines:

```
build selected model:  
    run_minimal_symmetry_tfm_experiment.py:434  
    run_minimal_symmetry_tfm_experiment.py:439  
  
fit on train only:  
    run_minimal_symmetry_tfm_experiment.py:448  
  
predict validation probabilities:  
    run_minimal_symmetry_tfm_experiment.py:452  
  
predict test probabilities:  
    run_minimal_symmetry_tfm_experiment.py:453  
  
choose threshold:  
    run_minimal_symmetry_tfm_experiment.py:456  
  
create binary predictions:  
    run_minimal_symmetry_tfm_experiment.py:461
```

The key line is:

```
model.fit(X_train_in, y_train)
```

Only `X_train_in` and `y_train` are passed to `fit`.

Validation and test are not merged into training.

## 2.6 Threshold Tuning

Function:

```
best_f1_threshold()  
line: run_minimal_symmetry_tfm_experiment.py:251
```

This function is used only when:

```
--threshold-mode tune_validation_f1
```

It searches candidate thresholds and chooses the one with highest validation F1.

The chosen threshold is then applied to test probabilities.

This is important for sparse **supervised\_by**, because the positive rate is low.

With fixed threshold **0.5**, several models predict no positives.

## 2.7 Metric Computation

Function:

```
evaluate_predictions()  
line: run_minimal_symmetry_tfm_experiment.py:265
```

This computes:

```
accuracy  
F1  
precision  
recall  
positive_rate_true  
positive_rate_pred  
average_precision  
roc_auc
```

Important detail:

```
accuracy/F1/precision/recall use thresholded predictions  
average_precision/roc_auc use probabilities
```

Therefore, a model can have:

```
F1 = 0  
ROC AUC > 0.8
```

This happened for fixed-threshold sparse **supervised\_by** before threshold tuning.

## 2.8 Output Writing

Function:

```
main()
line: run_minimal_symmetry_tfm_experiment.py:496
```

Important lines:

```
split rows:
    run_minimal_symmetry_tfm_experiment.py:502
    run_minimal_symmetry_tfm_experiment.py:503
    run_minimal_symmetry_tfm_experiment.py:504

make feature frames:
    run_minimal_symmetry_tfm_experiment.py:529
    run_minimal_symmetry_tfm_experiment.py:530
    run_minimal_symmetry_tfm_experiment.py:531

extract labels:
    run_minimal_symmetry_tfm_experiment.py:532
    run_minimal_symmetry_tfm_experiment.py:533
    run_minimal_symmetry_tfm_experiment.py:534

run model:
    run_minimal_symmetry_tfm_experiment.py:540

evaluate:
    run_minimal_symmetry_tfm_experiment.py:550

write metrics:
    run_minimal_symmetry_tfm_experiment.py:569

write predictions:
    run_minimal_symmetry_tfm_experiment.py:570
```

Each row in a TFM `metrics.csv` comes from:

```
fit_predict_one_model()
then evaluate_predictions()
then metrics_df.to_csv()
```

### 3. KGE Runner Code Path

File:

```
run_kge_relation_baselines.py
```

This script runs:

```
complex  
transr  
transe  
distmult
```

It uses the same model-facing CSV schema.

### 3.1 Data Loading

Function:

```
load_dataset_rows()  
line: run_kge_relation_baselines.py:142
```

This is parallel to the TFM loader.

It requires:

```
entity_1  
entity_2  
label  
split
```

It rejects hidden/leakage columns, including:

```
hidden_group_id  
hidden_seniority_rank  
role  
rank
```

### 3.2 Entity Encoding

Function:

```
build_entity_mapping()  
line: run_kge_relation_baselines.py:183
```

This creates:

```
entity_id -> integer index
```

Function:

```
encode_rows()  
line: run_kge_relation_baselines.py:188
```

This converts each row:

```
entity_1, entity_2, label
```

into:

```
[entity_1_index, entity_2_index], label
```

### 3.3 KGE Model Definitions

Function:

```
build_model()  
line: run_kge_relation_baselines.py:229
```

This contains all KGE architectures.

#### **DistMult**

Forward function:

```
run_kge_relation_baselines.py:239
```

Score:

```
score(A, B) = sum(e_A * r * e_B) + bias
```

Why it matters:

```
score(A, B) = score(B, A)
```

because elementwise multiplication is commutative.

Expected behavior:

```
good on symmetric relations  
bad on directed antisymmetric relations
```

## Complex

Forward function:

```
run_kge_relation_baselines.py:257
```

Score uses real and imaginary parts.

Why it matters:

```
Complex can represent asymmetric relations in theory.
```

In our implementation, however, it is trained with supervised BCE on explicit labels, not with the canonical ranking objective.

## TransE

Forward function:

```
run_kge_relation_baselines.py:282
```

Score:

```
score(A, B) = scale * (margin - distance(e_A + r, e_B)) + bias
```

Why it matters:

```
directional relation is represented by translation vector r
```

For symmetry, it can set:

```
r approximately 0
```

Then the score becomes mostly distance-based:

```
score(A, B) approximately -distance(e_A, e_B)
```

For total-order antisymmetry, it can learn a ranking direction.

For sparse **supervised\_by**, it struggles more because the relation is not a simple global order.

## TransR

Forward function:

```
run_kge_relation_baselines.py:310
```

Score:

```
score(A, B) = scale * (margin - distance(M e_A + r, M e_B)) + bias
```

It is like TransE, but first projects entities into relation space.

## 3.4 KGE Training

Function:

```
fit_predict_one_model()  
line: run_kge_relation_baselines.py:340
```

Important lines:

```
build model:  
    run_kge_relation_baselines.py:359  
  
optimizer:  
    run_kge_relation_baselines.py:360  
  
loss:  
    run_kge_relation_baselines.py:361  
  
convert train arrays to tensors:  
    run_kge_relation_baselines.py:363  
  
training loop:  
    run_kge_relation_baselines.py:375
```

```
forward pass:
    run_kge_relation_baselines.py:382

loss computation:
    run_kge_relation_baselines.py:383

backpropagation:
    run_kge_relation_baselines.py:384

optimizer step:
    run_kge_relation_baselines.py:387
```

The loss is:

```
BCEWithLogitsLoss
```

This means the KGE models are trained as supervised binary classifiers.

This is different from canonical knowledge-graph completion training, which often uses corrupted triples and ranking losses.

### 3.5 KGE Validation, Prediction, And Threshold

Validation loss:

```
run_kge_relation_baselines.py:389
run_kge_relation_baselines.py:391
run_kge_relation_baselines.py:392
```

Best checkpoint:

```
run_kge_relation_baselines.py:396
run_kge_relation_baselines.py:399
```

Prediction:

```
run_kge_relation_baselines.py:410
run_kge_relation_baselines.py:413
run_kge_relation_baselines.py:414
```

Convert logits to probabilities:

```
run_kge_relation_baselines.py:415  
run_kge_relation_baselines.py:416
```

Threshold:

```
run_kge_relation_baselines.py:419
```

Binary predictions:

```
run_kge_relation_baselines.py:424
```

### 3.6 KGE Metric Output

Main function:

```
run_kge_relation_baselines.py:497
```

Important lines:

```
split rows:  
    run_kge_relation_baselines.py:503  
    run_kge_relation_baselines.py:504  
    run_kge_relation_baselines.py:505  
  
encode rows:  
    run_kge_relation_baselines.py:520  
    run_kge_relation_baselines.py:521  
    run_kge_relation_baselines.py:522  
  
run model:  
    run_kge_relation_baselines.py:528  
  
evaluate:  
    run_kge_relation_baselines.py:539  
  
write metrics:  
    run_kge_relation_baselines.py:583  
  
write predictions:  
    run_kge_relation_baselines.py:584
```

Each KGE **metrics.csv** row comes from:

```
fit_predict_one_model()  
then evaluate_predictions()  
then write_csv(metrics.csv)
```

## 4. Experiment A: Group-Based Symmetry

Files:

```
generator: generate_minimal_symmetry_dataset.py  
TFM runner: run_minimal_symmetry_tfm_experiment.py  
KGE runner: run_kge_relation_baselines.py  
TFM results: results_symmetry_tfms/metrics.csv  
KGE results: results_symmetry_kge/metrics.csv
```

### 4.1 Generator Call Graph

Entry point:

```
generate_minimal_symmetry_dataset.py:357
```

Call sequence:

```
main()  
    parse_group_sizes()  
    build_entities()  
    build_unordered_pairs()  
    sample_pairs()  
    assign_symmetry_split()  
    validate_dataset()  
    write_csv(symmetry_friendship_atoms.csv)  
    write_csv(entities.csv)  
    write_csv(pair_assignments.csv)  
    write_json(metadata.json)
```

### 4.2 Entity Generation

Function:

```
build_entities()  
line: generate_minimal_symmetry_dataset.py:99
```

This creates hidden groups.

Important code concept:

```
hidden_group_id is stored in audit data  
exported entity IDs are shuffled
```

The model-facing CSV does not receive `hidden_group_id`.

### 4.3 Label Generation

Function:

```
build_unordered_pairs()  
line: generate_minimal_symmetry_dataset.py:136
```

Important line:

```
generate_minimal_symmetry_dataset.py:146
```

Logic:

```
label = 1 if left.hidden_group_id == right.hidden_group_id  
label = 0 otherwise
```

This creates symmetry:

```
same_group(A, B) = same_group(B, A)
```

It also creates transitivity:

```
if A and B are same group  
and B and C are same group  
then A and C are same group
```

### 4.4 Negative Sampling

Function:

```
sample_pairs()  
line: generate_minimal_symmetry_dataset.py:155
```

The generator keeps all positive pairs and samples negatives according to:

```
negative_count = len(positive_pairs) * negative_ratio
```

Important lines:

```
generate_minimal_symmetry_dataset.py:165  
generate_minimal_symmetry_dataset.py:167
```

With default `negative_ratio=1.0`, the dataset is balanced.

## 4.5 Split Generation

Function:

```
assign_symmetry_split()  
line: generate_minimal_symmetry_dataset.py:204
```

For each unordered pair, the generator creates two directed rows:

```
(A, B)  
(B, A)
```

Important lines:

```
choose train direction randomly:  
    generate_minimal_symmetry_dataset.py:230  
  
set pair role:  
    generate_minimal_symmetry_dataset.py:238  
  
set reverse split:  
    generate_minimal_symmetry_dataset.py:239  
  
write train row:  
    generate_minimal_symmetry_dataset.py:240
```

```
write reverse row:  
generate_minimal_symmetry_dataset.py:248
```

For demo pairs:

```
(A, B) train  
(B, A) train
```

For query pairs:

```
(A, B) train  
(B, A) validation/test
```

## 4.6 Validation

Function:

```
validate_dataset()  
line: generate_minimal_symmetry_dataset.py:275
```

This checks:

```
every directed row has reverse row  
reverse label is the same  
train/test are non-empty  
forbidden columns are absent
```

## 4.7 Result Rows And Code Path

TFM result file:

```
results_symmetry_tfms/metrics.csv
```

Rows:

```
TabPFN: accuracy 0.427, F1 0.530, ROC AUC 0.366  
TabICL: accuracy 0.438, F1 0.413, ROC AUC 0.380  
DistMult: accuracy 1.000, F1 1.000, ROC AUC 1.000
```

How each row was computed:

```
load_dataset_rows()
make_feature_frame()
fit_predict_one_model()
evaluate_predictions()
metrics_df.to_csv()
```

KGE result file:

```
results_symmetry_kge/metrics.csv
```

Rows:

```
Complex: accuracy 0.500, F1 0.547, ROC AUC 0.535
TransR:  accuracy 1.000, F1 1.000, ROC AUC 1.000
TransE:  accuracy 1.000, F1 1.000, ROC AUC 1.000
DistMult: accuracy 1.000, F1 1.000, ROC AUC 1.000
```

How each row was computed:

```
load_dataset_rows()
build_entity_mapping()
encode_rows()
fit_predict_one_model()
evaluate_predictions()
write_csv(metrics.csv)
```

## 4.8 Code-Level Interpretation

The source of the result is not only the model. It is the combination of:

```
label rule: hidden group equality
split rule: reverse pair held out or demonstrated
model inductive bias: categorical TFM vs embedding geometry
```

TransE/TransR/DistMult succeed because the generator creates a relation that can be represented geometrically.

The TFMs fail because `entity_1` and `entity_2` are arbitrary categorical tokens. The runner gives them no explicit graph structure, no group information, and no engineered reverse-pair feature.

## 5. Experiment B: Non-Transitive Symmetry

Files:

```
generator: generate_nontransitive_symmetry_dataset.py
KGE runner: run_kge_relation_baselines.py
KGE results: results_non_transitive_symmetry_kge/metrics.csv
```

No TFM result folder for this dataset is present in the current workspace.

### 5.1 Generator Call Graph

Entry point:

```
generate_nontransitive_symmetry_dataset.py:420
```

Call sequence:

```
main()
  build_entities()
  generate_positive_edges()
    sample_positive_edges()
    count_transitivity_violations()
  build_labeled_pairs()
  assign_symmetry_split()
  validate_dataset()
  write_csv(nontransitive_symmetry_atoms.csv)
  write_csv(entities.csv)
  write_csv(positive_edges.csv)
  write_csv(pair_assignments.csv)
  write_json(metadata.json)
```

### 5.2 Entity Generation

Function:

```
build_entities()
line: generate_nontransitive_symmetry_dataset.py:83
```

Entities have no hidden group.

The exported IDs are shuffled.

There is no cluster variable.

## 5.3 Positive Edge Generation

Function:

```
sample_positive_edges()  
line: generate_nontransitive_symmetry_dataset.py:106
```

This samples undirected graph edges.

If an unordered edge  $\{A, B\}$  is sampled:

```
label(A, B) = 1  
label(B, A) = 1
```

## 5.4 Non-Transitivity Check

Function:

```
count_transitivity_violations()  
line: generate_nontransitive_symmetry_dataset.py:130
```

This counts triples:

```
R(A, B) = 1  
R(B, C) = 1  
R(A, C) = 0
```

Important lines:

```
neighbors around middle entity:  
    generate_nontransitive_symmetry_dataset.py:141  
  
check whether closing edge is missing:  
    generate_nontransitive_symmetry_dataset.py:145
```

Current metadata:

```
transitivity violations = 592
```

Therefore, the dataset is not a hidden-group equivalence relation.

## 5.5 Label Generation

Function:

```
build_labeled_pairs()  
line: generate_nontransitive_symmetry_dataset.py:158
```

Important logic:

```
if unordered pair is a positive graph edge:  
    label = 1  
else:  
    label = 0
```

Code references:

```
generate_nontransitive_symmetry_dataset.py:172  
generate_nontransitive_symmetry_dataset.py:173  
generate_nontransitive_symmetry_dataset.py:176
```

## 5.6 Split Generation

Function:

```
assign_symmetry_split()  
line: generate_nontransitive_symmetry_dataset.py:217
```

The logic is the same as the group-based symmetry generator:

```
one direction is train  
reverse direction is train/validation/test depending on pair role
```

In the current metadata:

```
split_mode = reverse_holdout
```

Therefore:

```
all pairs are query pairs
one direction in train
reverse direction in validation/test
```

## 5.7 Validation

Function:

```
validate_dataset()
line: generate_nontransitive_symmetry_dataset.py:289
```

This validates:

```
required columns exist
every row has a reverse
reverse labels match
transitivity violations > 0
train/test are non-empty
```

The key condition is:

```
transitivity_violations > 0
```

Code reference:

```
generate_nontransitive_symmetry_dataset.py:340
```

## 5.8 Result Rows And Code Path

KGE result file:

```
results_non_transitive_symmetry_kge/metrics.csv
```

Rows:

```
Complex: accuracy 0.523, F1 0.470, ROC AUC 0.541
TransR:  accuracy 1.000, F1 1.000, ROC AUC 1.000
```

```
TransE: accuracy 1.000, F1 1.000, ROC AUC 1.000  
DistMult: accuracy 1.000, F1 1.000, ROC AUC 1.000
```

The KGE code path is:

```
load_dataset_rows()  
build_entity_mapping()  
encode_rows()  
build_model()  
fit_predict_one_model()  
evaluate_predictions()  
write_csv(metrics.csv)
```

## 5.9 Code-Level Interpretation

This experiment removed the transitive group structure.

However, the split is **reverse\_holdout**.

That means every test row is the reverse direction of a train row:

```
train: (A, B), label  
test:  (B, A), same label
```

For DistMult, the forward score is exactly symmetric:

```
run_kge_relation_baselines.py:239
```

For TransE and TransR, the model can learn relation vector close to zero, making the score mostly distance-based.

Distance is symmetric:

```
distance(A, B) = distance(B, A)
```

So this result is expected.

It means:

```
the dataset is non-transitive  
but the reverse-holdout task is still naturally solved by symmetric or  
near-symmetric KGE scoring
```

## 6. Experiment C: Total-Order Antisymmetry

Files:

```
generator: generate_minimal_antisymmetry_dataset.py
TFM runner: run_minimal_symmetry_tfm_experiment.py
KGE runner: run_kge_relation_baselines.py
TFM results: results_antisymmetry_tfms/metrics.csv
KGE results: results_antisymmetry_kge/metrics.csv
```

### 6.1 Generator Call Graph

Entry point:

```
generate_minimal_antisymmetry_dataset.py:333
```

Call sequence:

```
main()
  build_authors()
  build_unordered_pairs()
  sample_pairs()
  assign_antisymmetry_split()
    directed_labels()
  validate_dataset()
  write_csv(antisymmetry_coauthor_atoms.csv)
  write_csv(entities.csv)
  write_csv(pair_assignments.csv)
  write_json(metadata.json)
```

### 6.2 Entity And Rank Generation

Function:

```
build_authors()
line: generate_minimal_antisymmetry_dataset.py:79
```

This creates hidden ranks:

```
hidden_seniority_rank = 1, 2, ..., N
```

Important lines:

```
generate_minimal_antisymmetry_dataset.py:88  
generate_minimal_antisymmetry_dataset.py:91
```

Exported entity IDs are shuffled:

```
generate_minimal_antisymmetry_dataset.py:95  
generate_minimal_antisymmetry_dataset.py:97
```

Therefore, entity ID numbers do not directly reveal rank.

But the hidden relation is still a total order.

### 6.3 Pair Construction

Function:

```
build_unordered_pairs()  
line: generate_minimal_antisymmetry_dataset.py:111
```

This creates unordered pairs and stores both hidden ranks internally for label generation.

The model-facing CSV will not include the ranks.

### 6.4 Label Generation

Function:

```
directed_labels()  
line: generate_minimal_antisymmetry_dataset.py:161
```

Important logic:

```
if left_rank < right_rank:  
    left -> right = 1  
    right -> left = 0  
else:  
    right -> left = 1  
    left -> right = 0
```

So every unordered pair has exactly one positive direction.

This is the key difference from sparse **supervised\_by**.

In total-order antisymmetry:

```
for every pair A,B:  
    either A -> B is positive  
    or B -> A is positive
```

In sparse **supervised\_by**:

```
many pairs are negative both ways
```

## 6.5 Split Generation

Function:

```
assign_antisymmetry_split()  
line: generate_minimal_antisymmetry_dataset.py:172
```

Important lines:

```
create positive and negative direction:  
    generate_minimal_antisymmetry_dataset.py:198  
  
randomly choose train direction:  
    generate_minimal_antisymmetry_dataset.py:199  
  
set reverse split:  
    generate_minimal_antisymmetry_dataset.py:208  
  
write train row:  
    generate_minimal_antisymmetry_dataset.py:211  
  
write reverse row:  
    generate_minimal_antisymmetry_dataset.py:219
```

For demo pairs:

```
both directions are train
```

For query pairs:

```
one direction is train
```

```
opposite-label reverse direction is validation/test
```

## 6.6 Validation

Function:

```
validate_dataset()  
line: generate_minimal_antisymmetry_dataset.py:250
```

This checks that:

```
every directed row has a reverse  
reverse labels are different  
train/test are non-empty  
hidden rank columns are absent from model-facing table
```

## 6.7 Result Rows And Code Path

TFM result file:

```
results_antisymmetry_tfms/metrics.csv
```

Rows:

```
TabPFN: accuracy 0.625, F1 0.640, ROC AUC 0.722  
TabICL: accuracy 0.635, F1 0.653, ROC AUC 0.728  
DistMult: accuracy 0.323, F1 0.000, ROC AUC 0.085
```

KGE result file:

```
results_antisymmetry_kge/metrics.csv
```

Rows:

```
Complex: accuracy 0.521, F1 0.511, ROC AUC 0.517  
TransR:  accuracy 0.979, F1 0.981, ROC AUC 0.987  
TransE:  accuracy 0.979, F1 0.981, ROC AUC 0.997  
DistMult: accuracy 0.333, F1 0.179, ROC AUC 0.325
```

## 6.8 Code-Level Interpretation

The high TransE/TransR scores come from the generator logic.

The generator creates a global hidden ranking:

```
generate_minimal_antisymmetry_dataset.py:88
```

Then labels are derived from rank comparison:

```
generate_minimal_antisymmetry_dataset.py:167
```

TransE and TransR are directional embedding models:

```
TransE forward: run_kge_relation_baselines.py:282  
TransR forward: run_kge_relation_baselines.py:310
```

They can learn a direction in embedding space corresponding to rank/seniority.

Therefore, this result should be interpreted as:

```
TransE/TransR solve a structured ranking antisymmetry task.
```

Not as:

```
TransE/TransR solve arbitrary antisymmetry.
```

DistMult fails because:

```
DistMult forward is symmetric: run_kge_relation_baselines.py:239
```

It cannot naturally represent opposite labels for reversed pairs.

## 7. Experiment D: Sparse supervised\_by

Files:

```
generator: generate_sparse_supervised_by_dataset.py  
TFM runner: run_minimal_symmetry_tfm_experiment.py  
KGE runner: run_kge_relation_baselines.py
```

```
TFM results: results_sparse_supervised_by_tfms/metrics.csv
KGE results: results_sparse_kge/metrics.csv
```

This is the most important experiment for the current research story.

## 7.1 Generator Call Graph

Entry point:

```
generate_sparse_supervised_by_dataset.py:474
```

Call sequence:

```
main()
  build_entities()
  sample_supervision_edges()
  build_pair_records()
  assign_supervision_split()
  validate_dataset()
  write_csv(supervised_by_atoms.csv)
  write_csv(entities.csv)
  write_csv(positive_supervisions.csv)
  write_csv(pair_assignments.csv)
  write_json(metadata.json)
```

## 7.2 Entity And Role Generation

Function:

```
build_entities()
line: generate_sparse_supervised_by_dataset.py:77
```

This creates hidden roles:

```
student
supervisor
```

Important lines:

```
students:
    generate_sparse_supervised_by_dataset.py:89

supervisors:
```

```
generate_sparse_supervised_by_dataset.py:96
```

random exported IDs:

```
generate_sparse_supervised_by_dataset.py:104
```

```
generate_sparse_supervised_by_dataset.py:105
```

The role is written to `entities.csv` only.

It is not written to `supervised_by_atoms.csv`.

### 7.3 Positive Supervision Edge Sampling

Function:

```
sample_supervision_edges()
```

```
line: generate_sparse_supervised_by_dataset.py:125
```

This assigns each student one or more supervisors.

Important lines:

```
students list:
```

```
generate_sparse_supervised_by_dataset.py:137
```

```
supervisors list:
```

```
generate_sparse_supervised_by_dataset.py:138
```

```
sample supervisor count:
```

```
generate_sparse_supervised_by_dataset.py:144
```

```
add positive edge:
```

```
generate_sparse_supervised_by_dataset.py:146
```

Each positive edge is directed:

```
student -> supervisor
```

### 7.4 Positive And Negative Pair Records

Function:

```
build_pair_records()
```

```
line: generate_sparse_supervised_by_dataset.py:151
```

This is the most important function in the sparse generator.

It creates two kinds of unordered pair records:

```
supervision_edge  
unrelated_pair
```

For supervision edges:

```
student -> supervisor = 1  
supervisor -> student = 0
```

Code references:

```
generate_sparse_supervised_by_dataset.py:170  
generate_sparse_supervised_by_dataset.py:171  
generate_sparse_supervised_by_dataset.py:172  
generate_sparse_supervised_by_dataset.py:173
```

If the positive edge appears in the opposite unordered orientation, the labels are flipped:

```
generate_sparse_supervised_by_dataset.py:176  
generate_sparse_supervised_by_dataset.py:177  
generate_sparse_supervised_by_dataset.py:178
```

For unrelated pairs:

```
A -> B = 0  
B -> A = 0
```

Code references:

```
generate_sparse_supervised_by_dataset.py:195  
generate_sparse_supervised_by_dataset.py:200  
generate_sparse_supervised_by_dataset.py:201
```

This is the key methodological improvement over total-order antisymmetry.

## 7.5 Negative Ratio

Still inside:

```
build_pair_records()  
line: generate_sparse_supervised_by_dataset.py:151
```

The negative-ratio logic is:

```
sample negative unrelated pairs = positive supervision edges *  
negative_ratio
```

Code references:

```
generate_sparse_supervised_by_dataset.py:208  
generate_sparse_supervised_by_dataset.py:209  
generate_sparse_supervised_by_dataset.py:210
```

With `negative_ratio=3.0`, if there are 68 positive supervision edges:

```
unrelated unordered pairs = 68 * 3 = 204
```

Each supervision edge contributes:

```
one positive row  
one reverse-negative row
```

Each unrelated pair contributes:

```
two negative rows
```

So:

```
positive rows = 68  
total rows = 68 * 2 + 204 * 2 = 544  
positive rate = 68 / 544 = 0.125
```

This explains the imbalance.

## 7.6 Split Generation

Function:

```
assign_supervision_split()  
line: generate_sparse_supervised_by_dataset.py:252
```

Important lines:

```
choose demo/query indices:  
    generate_sparse_supervised_by_dataset.py:263  
  
choose validation query rows:  
    generate_sparse_supervised_by_dataset.py:272  
  
read labels for both directions:  
    generate_sparse_supervised_by_dataset.py:279  
    generate_sparse_supervised_by_dataset.py:280  
    generate_sparse_supervised_by_dataset.py:281  
    generate_sparse_supervised_by_dataset.py:282  
  
randomly choose which direction is train:  
    generate_sparse_supervised_by_dataset.py:284  
  
set reverse split:  
    generate_sparse_supervised_by_dataset.py:300  
  
write train row:  
    generate_sparse_supervised_by_dataset.py:303  
  
write reverse row:  
    generate_sparse_supervised_by_dataset.py:311
```

In **reverse\_holdout** mode:

```
one direction of each pair is train  
the reverse direction is validation/test
```

In **supervision\_demonstration** mode:

```
some pairs have both directions in train as demonstrations  
remaining query pairs use reverse holdout
```

## 7.7 Validation

Function:

```
validate_dataset()  
line: generate_sparse_supervised_by_dataset.py:343
```

This checks:

```
model-facing columns are exactly safe  
every row has reverse row  
there are no both-positive pairs  
there are supervision edges with positive one way and negative reverse  
there are both-negative unrelated pairs  
train and test contain both labels
```

Important lines:

```
forbidden columns:  
    generate_sparse_supervised_by_dataset.py:346  
  
reverse lookup:  
    generate_sparse_supervised_by_dataset.py:375  
  
both-positive violation check:  
    generate_sparse_supervised_by_dataset.py:380  
  
positive reverse-negative count:  
    generate_sparse_supervised_by_dataset.py:382  
  
both-negative pair count:  
    generate_sparse_supervised_by_dataset.py:388  
  
validity condition:  
    generate_sparse_supervised_by_dataset.py:419
```

## 7.8 TFM Result Rows And Code Path

TFM result file:

```
results_sparse_supervised_by_tfms/metrics.csv
```

The metadata says:

```
threshold_mode = tune_validation_f1  
train rows = 272
```

```
validation rows = 54  
test rows = 218
```

Rows:

```
TabPFN:  
  threshold = 0.112  
  accuracy = 0.592  
  F1 = 0.276  
  precision = 0.179  
  recall = 0.607  
  AP = 0.245  
  ROC AUC = 0.652
```

```
TabICL:  
  threshold = 0.127  
  accuracy = 0.798  
  F1 = 0.542  
  precision = 0.382  
  recall = 0.929  
  AP = 0.483  
  ROC AUC = 0.894
```

How the TabICL row was produced:

```
load supervised_by_atoms.csv:  
  run_minimal_symmetry_tfm_experiment.py:145  
  
make categorical features:  
  run_minimal_symmetry_tfm_experiment.py:183  
  
build TabICL:  
  run_minimal_symmetry_tfm_experiment.py:219  
  
fit TabICL:  
  run_minimal_symmetry_tfm_experiment.py:448  
  
predict validation and test probabilities:  
  run_minimal_symmetry_tfm_experiment.py:452  
  run_minimal_symmetry_tfm_experiment.py:453  
  
tune threshold on validation F1:  
  run_minimal_symmetry_tfm_experiment.py:251  
  run_minimal_symmetry_tfm_experiment.py:456  
  
threshold test probabilities:  
  run_minimal_symmetry_tfm_experiment.py:461  
  
compute test metrics:
```

```
run_minimal_symmetry_tfm_experiment.py:265
run_minimal_symmetry_tfm_experiment.py:550

write_metrics.csv:
run_minimal_symmetry_tfm_experiment.py:569
```

## 7.9 KGE Result Rows And Code Path

KGE result file:

```
results_sparse_kge/metrics.csv
```

The metadata says:

```
threshold_mode = tune_validation_f1
embedding_dim = 64
relation_dim = 64
epochs = 1000
distance = l2
```

Rows:

```
Complex:
  threshold = 0.440
  accuracy = 0.628
  F1 = 0.198
  precision = 0.137
  recall = 0.357
  AP = 0.226
  ROC AUC = 0.616

TransR:
  threshold = 0.160
  accuracy = 0.766
  F1 = 0.215
  precision = 0.189
  recall = 0.250
  AP = 0.194
  ROC AUC = 0.639

TransE:
  threshold = 0.086
  accuracy = 0.610
  F1 = 0.388
  precision = 0.243
  recall = 0.964
  AP = 0.325
```

```
ROC AUC = 0.826
```

```
DistMult:
```

```
threshold = 0.230
```

```
accuracy = 0.417
```

```
F1 = 0.181
```

```
precision = 0.110
```

```
recall = 0.500
```

```
AP = 0.111
```

```
ROC AUC = 0.426
```

How the TransE row was produced:

```
load supervised_by_atoms.csv:
```

```
run_kge_relation_baselines.py:142
```

```
build entity index mapping:
```

```
run_kge_relation_baselines.py:183
```

```
encode rows:
```

```
run_kge_relation_baselines.py:188
```

```
build TransE model:
```

```
run_kge_relation_baselines.py:327
```

```
TransE forward score:
```

```
run_kge_relation_baselines.py:282
```

```
train with BCE:
```

```
run_kge_relation_baselines.py:375
```

```
run_kge_relation_baselines.py:382
```

```
run_kge_relation_baselines.py:383
```

```
predict probabilities:
```

```
run_kge_relation_baselines.py:413
```

```
run_kge_relation_baselines.py:416
```

```
tune threshold:
```

```
run_kge_relation_baselines.py:197
```

```
run_kge_relation_baselines.py:419
```

```
compute test metrics:
```

```
run_kge_relation_baselines.py:211
```

```
run_kge_relation_baselines.py:539
```

```
write metrics.csv:
```

```
run_kge_relation_baselines.py:583
```

## 7.10 Code-Level Interpretation

This is the clearest experiment because the generator is not creating a global total order.

The important code difference is:

```
total-order antisymmetry:
    every unordered pair has one positive direction
    generate_minimal_antisymmetry_dataset.py:161

sparse supervised_by:
    supervision edges have one positive direction
    unrelated pairs are negative both directions
    generate_sparse_supervised_by_dataset.py:170
    generate_sparse_supervised_by_dataset.py:195
```

That changes what models can exploit.

TransE and TransR no longer get a simple global ranking relation.

TabICL performs best because it ranks positives well in this sparse table:

```
ROC AUC = 0.894
AP = 0.483
F1 = 0.542
```

The F1 is produced only after validation threshold tuning.

Without threshold tuning, fixed threshold **0.5** made several models predict almost no positives.

## 8. How To Explain Each Result From Code

### 8.1 Why TFMs Fail On Group-Based Symmetry

Code facts:

```
label is hidden group equality:
    generate_minimal_symmetry_dataset.py:146

features are only entity_1/entity_2:
    run_minimal_symmetry_tfm_experiment.py:183

TFMs receive categorical entity IDs:
    run_minimal_symmetry_tfm_experiment.py:193
    run_minimal_symmetry_tfm_experiment.py:219
```

Explanation:

The model does not see group ID.  
The model sees arbitrary entity tokens.  
The relation requires reconstructing hidden groups from pair examples.  
The TFMs do not do this reliably.

## 8.2 Why KGE Models Solve Group-Based Symmetry

Code facts:

```
KGE models learn entity embeddings:  
run_kge_relation_baselines.py:230  
run_kge_relation_baselines.py:270  
run_kge_relation_baselines.py:289  
  
TransE/TransR use distances:  
run_kge_relation_baselines.py:285  
run_kge_relation_baselines.py:315  
  
DistMult is symmetric:  
run_kge_relation_baselines.py:239
```

Explanation:

The group dataset is cluster-like.  
Embedding models can place same-group entities close together.  
DistMult also gives identical score to reversed pairs.

## 8.3 Why Non-Transitive Symmetry Still Gives KGE 100 Percent

Code facts:

```
positive edges are random graph edges:  
generate_nontransitive_symmetry_dataset.py:106  
  
transitivity violations are checked:  
generate_nontransitive_symmetry_dataset.py:130  
  
split is reverse_holdout:  
generate_nontransitive_symmetry_dataset.py:217
```

Explanation:

The dataset is non-transitive.  
But each test example is the reverse of a train example.

Symmetric or near-symmetric scores solve this directly.

## 8.4 Why TransE/TransR Solve Total-Order Antisymmetry

Code facts:

```
hidden rank is assigned:
    generate_minimal_antisymmetry_dataset.py:88

label is rank comparison:
    generate_minimal_antisymmetry_dataset.py:167

TransE/TransR are directional translation models:
    run_kge_relation_baselines.py:282
    run_kge_relation_baselines.py:310
```

Explanation:

The dataset is a global ranking problem.  
Translation embeddings can encode ranking direction.  
This is easier than arbitrary antisymmetry.

## 8.5 Why DistMult Fails On Antisymmetry

Code fact:

```
DistMult score:
    run_kge_relation_baselines.py:239
```

Explanation:

$$\text{score}(A, B) = \text{score}(B, A)$$

But antisymmetry requires:

$$\begin{aligned} R(A, B) &= 1 \\ R(B, A) &= 0 \end{aligned}$$

So the model bias is wrong.

## 8.6 Why Sparse supervised\_by Is Different

### Code facts:

```
sample real directed supervision edges:  
    generate_sparse_supervised_by_dataset.py:125  
  
supervision edge labels:  
    generate_sparse_supervised_by_dataset.py:170  
  
unrelated pair labels:  
    generate_sparse_supervised_by_dataset.py:195
```

### Explanation:

The model cannot assume that every pair has one positive direction.  
Many pairs are negative both ways.  
This breaks the total-order shortcut.

## 8.7 Why TabICL Wins On Sparse supervised\_by

### Code facts:

```
TabICL fit:  
    run_minimal_symmetry_tfm_experiment.py:439  
    run_minimal_symmetry_tfm_experiment.py:448  
  
TabICL validation threshold:  
    run_minimal_symmetry_tfm_experiment.py:251  
    run_minimal_symmetry_tfm_experiment.py:456  
  
TabICL metrics:  
    run_minimal_symmetry_tfm_experiment.py:265
```

### Result facts:

```
TabICL threshold = 0.127  
F1 = 0.542  
recall = 0.929  
precision = 0.382  
AP = 0.483  
ROC AUC = 0.894
```

### Explanation:

TabICL ranks positives well and, after validation threshold tuning, retrieves most positives.  
The KGE models either overpredict positives or fail to rank sparse positives as well.

## 9. Exact Result-To-Code Map

### 9.1 `results_symmetry_tfms/metrics.csv`

Dataset:

```
minimal_symmetry_dataset/symmetry_friendship_atoms.csv
```

Generator functions:

```
build_entities()  
build_unordered_pairs()  
sample_pairs()  
assign_symmetry_split()  
validate_dataset()
```

Runner functions:

```
load_dataset_rows()  
make_feature_frame()  
encode_for_tabpfn()  
build_tabpfn_classifier()  
build_tabicl_classifier()  
fit_predict_one_model()  
best_f1_threshold()  
evaluate_predictions()  
main()
```

Main interpretation:

TFMs struggle on hidden-group symmetry.  
DistMult solves because its score is symmetric.

### 9.2 `results_symmetry_kge/metrics.csv`

Dataset:

```
minimal_symmetry_dataset/symmetry_friendship_atoms.csv
```

Generator functions:

```
same as 9.1
```

Runner functions:

```
load_dataset_rows()  
build_entity_mapping()  
encode_rows()  
build_model()  
fit_predict_one_model()  
best_f1_threshold()  
evaluate_predictions()  
main()
```

Main interpretation:

```
KGE geometry solves hidden group structure.
```

### 9.3 results\_non\_transitive\_symmetry\_kge/metrics.csv

Dataset:

```
nontransitive_symmetry_dataset/nontransitive_symmetry_atoms.csv
```

Generator functions:

```
build_entities()  
sample_positive_edges()  
count_transitivity_violations()  
build_labeled_pairs()  
assign_symmetry_split()  
validate_dataset()
```

Runner functions:

same KGE path as 9.2

Main interpretation:

Non-transitivity is validated, but reverse-holdout is still easy for symmetric KGE scoring.

## 9.4 `results_antisymmetry_tfms/metrics.csv`

Dataset:

`minimal_antisymmetry_dataset/antisymmetry_coauthor_atoms.csv`

Generator functions:

```
build_authors()
build_unordered_pairs()
sample_pairs()
directed_labels()
assign_antisymmetry_split()
validate_dataset()
```

Runner functions:

same TFM path as 9.1

Main interpretation:

TFMs are above chance on total-order antisymmetry.

## 9.5 `results_antisymmetry_kge/metrics.csv`

Dataset:

`minimal_antisymmetry_dataset/antisymmetry_coauthor_atoms.csv`

Generator functions:

same as 9.4

Runner functions:

same KGE path as 9.2

Main interpretation:

TransE/TransR solve the hidden ranking relation.  
DistMult fails because it is symmetric.

## 9.6 results\_sparse\_supervised\_by\_tfms/metrics.csv

Dataset:

sparse\_supervised\_by\_dataset/supervised\_by\_atoms.csv

Generator functions:

```
build_entities()  
sample_supervision_edges()  
build_pair_records()  
assign_supervision_split()  
validate_dataset()
```

Runner functions:

same TFM path as 9.1

Main interpretation:

TabICL is strongest on sparse supervised\_by.  
Threshold tuning is essential because positives are sparse.

## 9.7 results\_sparse\_kge/metrics.csv

Dataset:

```
sparse_supervised_by_dataset/supervised_by_atoms.csv
```

Generator functions:

```
same as 9.6
```

Runner functions:

```
same KGE path as 9.2
```

Main interpretation:

```
KGE baselines do not solve sparse supervised_by.  
TransE has high recall but poor precision.  
TabICL beats all KGE baselines by F1, AP, and ROC AUC.
```

## 10. What To Study First

Recommended reading order:

1. run\_minimal\_symmetry\_tfm\_experiment.py:496
2. run\_minimal\_symmetry\_tfm\_experiment.py:414
3. run\_minimal\_symmetry\_tfm\_experiment.py:265
4. run\_kge\_relation\_baselines.py:497
5. run\_kge\_relation\_baselines.py:340
6. run\_kge\_relation\_baselines.py:229
7. generate\_sparse\_supervised\_by\_dataset.py:151
8. generate\_sparse\_supervised\_by\_dataset.py:252
9. generate\_minimal\_antisymmetry\_dataset.py:161
10. generate\_nontransitive\_symmetry\_dataset.py:130

This order starts from the machine-learning computation and then moves backward to the data-generation assumptions.

## 11. Questions To Be Ready To Answer

If your supervisor asks "where is the data loaded?", answer:

```
TFM runner: run_minimal_symmetry_tfm_experiment.py:145  
KGE runner: run_kge_relation_baselines.py:142
```

If asked "where are train/validation/test separated?", answer:

```
TFM runner: run_minimal_symmetry_tfm_experiment.py:502
KGE runner: run_kge_relation_baselines.py:503
```

If asked "where are the TFM's fit?", answer:

```
run_minimal_symmetry_tfm_experiment.py:448
```

If asked "where are KGE models trained?", answer:

```
run_kge_relation_baselines.py:375
```

If asked "where is the sparse supervised\_by label created?", answer:

```
positive supervision edge:
    generate_sparse_supervised_by_dataset.py:170

unrelated both-negative pair:
    generate_sparse_supervised_by_dataset.py:195
```

If asked "why is sparse supervised\_by imbalanced?", answer:

```
negative unrelated pairs are sampled at 3 times the number of positive
supervision edges:
    generate_sparse_supervised_by_dataset.py:208
```

If asked "where is F1 threshold tuning done?", answer:

```
TFM: run_minimal_symmetry_tfm_experiment.py:251
KGE: run_kge_relation_baselines.py:197
```

If asked "why can TransE solve total-order antisymmetry?", answer:

```
the generator creates hidden rank:
    generate_minimal_antisymmetry_dataset.py:88

labels are rank comparisons:
    generate_minimal_antisymmetry_dataset.py:167
```

```
TransE learns a directional translation:  
run_kge_relation_baselines.py:282
```

If asked "why is non-transitive symmetry still easy for KGE?", answer:

```
the relation is non-transitive:  
generate_nontransitive_symmetry_dataset.py:130  
  
but the split is reverse-holdout:  
generate_nontransitive_symmetry_dataset.py:217  
  
and DistMult is symmetric:  
run_kge_relation_baselines.py:239
```

## 12. The Main Code-Oriented Claim

The most defensible code-level claim is:

```
The sparse supervised_by generator creates a genuinely sparse directed  
relation:  
    positive supervision edges have one positive direction;  
    unrelated pairs are negative both ways;  
    role information is excluded from the model-facing CSV.  
  
The TFM runner fits TabPFN/TabICL only on train rows and evaluates on test  
rows.  
  
With validation-F1 threshold tuning, TabICL obtains the best sparse  
supervised_by result among the tested models.
```

Everything in that claim is directly traceable to:

```
generate_sparse_supervised_by_dataset.py  
run_minimal_symmetry_tfm_experiment.py  
results_sparse_supervised_by_tfms/metrics.csv  
results_sparse_kge/metrics.csv
```

## 13. Code Audit

The syntax layer was checked by parsing the six main experiment scripts with Python `ast.parse`. This passed for `generate_minimal_symmetry_dataset.py`, `generate_minimal_antisymmetry_dataset.py`, `generate_nontransitive_symmetry_dataset.py`, `generate_sparse_supervised_by_dataset.py`, `run_minimal_symmetry_tfm_experiment.py`,

and `run_kge_relation_baselines.py`. This does not prove mathematical correctness, but it confirms the scripts are syntactically valid Python.

The data contract layer was checked in both runners. The TFM runner requires `entity_1`, `entity_2`, `label`, and `split` in `run_minimal_symmetry_tfm_experiment.py:153–157`. It also rejects known hidden group columns in `run_minimal_symmetry_tfm_experiment.py:158–160`. The KGE runner requires the same four columns in `run_kge_relation_baselines.py:150–154` and rejects hidden/leakage columns in `run_kge_relation_baselines.py:156–158`. The model-facing CSVs written by the generators contain only `entity_1`, `entity_2`, `label`, and `split`: group symmetry at `generate_minimal_symmetry_dataset.py:417`, non-transitive symmetry at `generate_nontransitive_symmetry_dataset.py:483`, total-order antisymmetry at `generate_minimal_antisymmetry_dataset.py:383`, and sparse supervised\_by at `generate_sparse_supervised_by_dataset.py:543`.

The split layer was checked explicitly. In the TFM runner, raw rows are separated into train, validation, and test in `run_minimal_symmetry_tfm_experiment.py:502–504`; the runtime dataframes are separated again in `run_minimal_symmetry_tfm_experiment.py:525–527`; features and targets are built from those split-specific frames in `run_minimal_symmetry_tfm_experiment.py:529–534`. In the KGE runner, rows are separated in `run_kge_relation_baselines.py:503–505`, then encoded separately in `run_kge_relation_baselines.py:520–522`. Validation and test rows are not merged into the fit set. For TFM, the fit call is `model.fit(X_train_in, y_train)` in `run_minimal_symmetry_tfm_experiment.py:448`. For KGEs, the training tensor is built from `train_pairs_np` and `y_train` in `run_kge_relation_baselines.py:363`.

The label-generation layer was checked separately for each dataset family. In the group-based symmetry generator, labels come from hidden group equality in `generate_minimal_symmetry_dataset.py:146`. In the non-transitive symmetry generator, positive undirected graph edges are sampled in `generate_nontransitive_symmetry_dataset.py:106–119`, positive and negative unordered labels are built in `generate_nontransitive_symmetry_dataset.py:158–180`, symmetric directed rows are assembled around `generate_nontransitive_symmetry_dataset.py:242–268`, and transitivity violations are counted in `generate_nontransitive_symmetry_dataset.py:130–155` and reported during validation at `generate_nontransitive_symmetry_dataset.py:321–324`. In the total-order antisymmetry generator, hidden seniority ranks are created in `generate_minimal_antisymmetry_dataset.py:88–106`, rank-based directed labels are assigned in `generate_minimal_antisymmetry_dataset.py:161–169`, and one direction is put in train while the reverse goes to validation/test in `generate_minimal_antisymmetry_dataset.py:197–208`. In the sparse supervised-by generator, hidden student/supervisor roles are created in `generate_sparse_supervised_by_dataset.py:77–114`, positive supervision edges are sampled in `generate_sparse_supervised_by_dataset.py:125–148`, reverse labels for supervision edges are created in `generate_sparse_supervised_by_dataset.py:168–194`, unrelated both-negative pairs are created in `generate_sparse_supervised_by_dataset.py:195–210`, and train/reverse split rows are assembled in `generate_sparse_supervised_by_dataset.py:277–317`.

The TFM computation layer was checked in `fit_predict_one_model`, starting at `run_minimal_symmetry_tfm_experiment.py:414`. The actual fit call is `model.fit(X_train_in, y_train)` in `run_minimal_symmetry_tfm_experiment.py:448`. Probabilities are obtained through `predict_positive_proba`, defined in `run_minimal_symmetry_tfm_experiment.py:247–248`, and

called for validation/test in `run_minimal_symmetry_tfm_experiment.py:452–453`. Threshold tuning is implemented in `best_f1_threshold` at `run_minimal_symmetry_tfm_experiment.py:251–262`, and metric computation is implemented in `evaluate_predictions` at `run_minimal_symmetry_tfm_experiment.py:265–280`.

The KGE computation layer was checked in `run_kge_relation_baselines.py`. The model factory is `build_model` at `run_kge_relation_baselines.py:229`. The scoring models are implemented as `DistMultModel` at `run_kge_relation_baselines.py:230–242`, `ComplExModel` at `run_kge_relation_baselines.py:244–268`, `TransEModel` at `run_kge_relation_baselines.py:270–287`, and `TransRModel` at `run_kge_relation_baselines.py:289–321`. Training is implemented in `fit_predict_one_model` at `run_kge_relation_baselines.py:340–430`, with `BCEWithLogitsLoss` at `run_kge_relation_baselines.py:361`, minibatch logits/loss at `run_kge_relation_baselines.py:382–383`, sigmoid probabilities at `run_kge_relation_baselines.py:413–416`, and validation-F1 threshold selection at `run_kge_relation_baselines.py:419–424`. Metric computation is implemented in `evaluate_predictions` at `run_kge_relation_baselines.py:211–226`.

The result-writing layer was checked to ensure reported metrics come from test predictions. In the TFM runner, test metrics are computed in `run_minimal_symmetry_tfm_experiment.py:550` and written to CSV in `run_minimal_symmetry_tfm_experiment.py:569`. In the KGE runner, test metrics are computed in `run_kge_relation_baselines.py:539` and written to CSV in `run_kge_relation_baselines.py:583`.

The local dry-run audit produced these checks:

- Group symmetry dry-run: `entities=60`, `train_rows=360`, `validation_rows=24`, `test_rows=96`, balanced train labels, and approximately balanced test labels.
- Total-order antisymmetry dry-run: `entities=60`, `train_rows=360`, `validation_rows=24`, `test_rows=96`, with both positive and negative examples in train and test.
- Non-transitive symmetry TFM dry-run: `entities=80`, `train_rows=320`, `validation_rows=64`, `test_rows=256`, balanced train labels, and approximately balanced test labels.
- Non-transitive symmetry KGE dry-run: same split counts as the TFM runner, confirming both runners consume the same dataset contract.
- Sparse supervised-by audit generation: `entities=60`, `supervision_edges=68`, `unrelated_pairs=204`, `rows=544`, `train=272`, `validation=54`, `test=218`, and `positive_rate=0.1250`.
- Sparse supervised-by TFM/KGE dry-runs: both runners saw `train_rows=272`, `validation_rows=54`, and `test_rows=218`, confirming consistent consumption of the generated file.